

A Reproducible Data Workflow for Exploring Survival in the Titanic Passenger Dataset

Vasileios Panagiotopoulos

Udacity, AI Mastery Capstone

AI Programming Foundations Project

October 6, 2026

A Reproducible Data Workflow for Exploring Survival in the Titanic Passenger Dataset

Overview

This project builds a reproducible data workflow in Python for the Titanic passenger dataset. The workflow loads the data, cleans it with documented functions, explores it, produces three visualizations and summarizes the findings in a Jupyter notebook (`data_workflow.ipynb`), supported by a README, a requirements.txt file and a GitHub repository. The data come from the seaborn-data repository (<https://github.com/mwaskom/seaborn-data/blob/master/titanic.csv>), which hosts a version of the Kaggle Titanic dataset (<https://www.kaggle.com/c/titanic/data>). The aim was to find out which passenger characteristics, namely sex, class, age and fare, are linked to survival.

Dataset Description

The dataset describes 891 passengers of the Titanic, with one row per passenger and 15 columns. The outcome variable is survived, coded 0 for passengers who died and 1 for those who survived; overall, 38.4% of passengers survived (Table 1 in the notebook). I focused on four explanatory variables: sex, passenger class (pclass, from 1 for first class to 3 for third class), age and the fare paid. Several other columns repeat the same information in a different form, such as class and pclass or alive and survived. The raw data also have gaps: deck is missing for 77.2% of passengers, age for 19.9% (177 passengers) and port of embarkation for two passengers. After cleaning, the dataset has 891 rows and 10 columns.

Workflow Description

The workflow follows a fixed sequence of steps, each written as code so it can be rerun from scratch. Ingestion reads the CSV file with pandas from a URL pinned to a specific commit of the source repository, so the input cannot change between runs. Cleaning runs a pipeline

function, `clean_titanic`, which removes redundant and mostly empty columns, fills in missing ages and fills in the two missing ports; a helper, `report_missing`, shows the missing values before and after cleaning.

Exploratory analysis uses three functions: `summarize_numeric` for summary statistics, `survival_rate_by` for the number of passengers and survival rate in any grouping, and `correlation_with_survival` for the numeric columns. Their outputs are labelled Tables 1 to 9 in the notebook. Visualization produces three Seaborn charts (Figures 1 to 3), which are also saved as image files, and the notebook ends with a written summary. This structure, moving from data preprocessing to description through exploration and visualization, using open tools such as Python, Jupyter and Markdown and clean, reusable code, reflects the workflow Danchev (2022) presents as the basis of transparent and reproducible data analysis.

Key Decisions and Assumptions

The first cleaning decision was to drop columns that repeat other columns (`class`, `alive`, `embark_town`, `who` and `adult_male`), which I confirmed with cross-tabulations before removing them, and to drop `deck` because more than three quarters of its values are missing. The most important decision concerned age. Simple approaches to missing data, such as replacing every missing value with one overall mean, usually produce biased results (Donders et al., 2006). Because age differs clearly between classes and between men and women, I filled each missing age with the median age of passengers of the same class and sex, and added an `age_was_missing` flag so imputed values can always be identified. The two missing ports were filled with the most common port, Southampton, since two values out of 891 barely affect the distribution.

I also chose not to remove 107 rows that look identical to another row. This version of the dataset has no names or passenger IDs, so these rows are most likely different people who

share the same details, and deleting them would have removed real passengers. In the exploratory analysis I focused on survival rates by group: because survived is coded 0 or 1, its mean is the share of passengers who survived.

Each plot was designed to answer one question. Figure 1 compares survival rates by class and sex, the two strongest patterns in the data. Figure 2 shows the age distribution of survivors and non-survivors using recorded ages only, so the imputed medians do not create a false peak. Figure 3 compares fares by class and outcome on a logarithmic scale, because a few very high fares would otherwise hide the differences; the 15 passengers with a fare of zero are excluded from this plot.

Results and Interpretation

Sex made the largest difference to survival. In every class, women survived far more often than men: 96.8% compared with 36.9% in first class, 92.1% compared with 15.7% in second class and 50.0% compared with 13.5% in third class (Table 4, Figure 1). Class also mattered, with survival falling from 63.0% in first class to 47.3% in second and 24.2% in third (Table 3). Third-class men were both the largest group, at 347 passengers, and the worst off, with only 13.5% surviving.

Age showed a weaker pattern. Children aged 0 to 11 were the only age group in which more passengers survived than died (57.4%), while passengers aged 60 and over had the lowest survival rate (26.9%) (Table 7, Figure 2). Fare is closely tied to class: the median fare was about 60 in first class, 14 in second and 8 in third (Table 6). Within first class, survivors had paid noticeably higher fares than those who died, but in second and third class the difference was small (Figure 3). Two results were unexpected: only half of third-class women survived despite

the women-first policy, and passengers aged 35 to 59 survived slightly more often than those aged 18 to 34.

Responsible Practice (Bias and Data Quality)

The main risk of bias comes from the missing ages. Ages are missing much more often in third class (27.7%) than in first (13.9%) or second class (6.0%) (Table 8), so any age-based result is least reliable for third-class passengers. Imputing with the median of the same class and sex reduces this risk compared with a single overall value, and Figure 2 uses recorded ages only. However, filling in a single value treats a guess as if it were known, which makes results look more certain than they are; multiple imputation would reflect this uncertainty better (Donders et al., 2006), and I would use it in a next step.

Other choices also limit what the analysis can show. Dropping deck removes information about cabin location, which may have affected access to the lifeboats, and the 15 zero fares were excluded from Figure 3 (Table 9). More importantly, the findings show association rather than cause. Sex, class, fare and age are linked to one another, so a pattern in one variable may reflect another; identifying and dealing with such confounding and with selection bias is part of responsible data analysis (Danchev, 2022). Finally, the dataset covers 891 passengers rather than everyone on board, so the results describe this sample.

Reproducibility

The project is designed so that someone else can rerun it and get the same results. The README explains the steps: clone the repository, create a virtual environment, install the dependencies with `pip install -r requirements.txt`, open `data_workflow.ipynb` and run all cells. The `requirements.txt` file records the exact package versions used (for example pandas 2.3.3, NumPy 2.0.2, Matplotlib 3.9.4 and Seaborn 0.13.2 with Python 3.9), the data are loaded from a

URL pinned to a fixed commit, and every cleaning step is done in code rather than by editing the data by hand. These choices follow established rules for reproducible computational research, including keeping track of how every result was produced, avoiding manual data manipulation, recording the exact versions of software used and keeping all scripts under version control (Sandve et al., 2013).

The work is tracked in a GitHub repository with a main branch and a develop branch, and changes were committed as each part of the project was completed. Keeping the code, its outputs and the written explanation together in a Jupyter notebook with Markdown also makes the analysis easier to follow and check (Danchev, 2022).

References

- Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education*, 5(56), 156. <https://doi.org/10.21105/jose.00156>
- Donders, A. R. T., van der Heijden, G. J. M. G., Stijnen, T., & Moons, K. G. M. (2006). Review: A gentle introduction to imputation of missing values. *Journal of Clinical Epidemiology*, 59(10), 1087–1091. <https://doi.org/10.1016/j.jclinepi.2006.01.014>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10), Article e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>